pfi

# Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

## Case Study: Boosting on Credit Card Default

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# When the models are this close

### Hands-on: 10.1 and 10.2

bootstrap_interval on a validation score, the paired difference, and compare_models with out-of-fold thresholds.

### Case study: six models

Fit, tune, and choose inside the training rows; report on the validation rows; describe; say where it fails.

**How much of the gap between two models is the models, and how much is which 6,000 accounts were scored?**

**The test rows stay closed: the case study is where they would usually be opened.**

# How the bootstrap interval works

### 1. Resample

the validation rows, with replacement

### 2. Score again

the same fitted model; nothing is refitted

### 3. Repeat 1,000 times

one ROC AUC per resample

### 4. Middle 95 percent

is the interval

### You control

The number of resamples and the seed.

### What comes out

An interval around each ROC AUC, and around a difference of two models on the same resamples.

# How much of a difference is noise

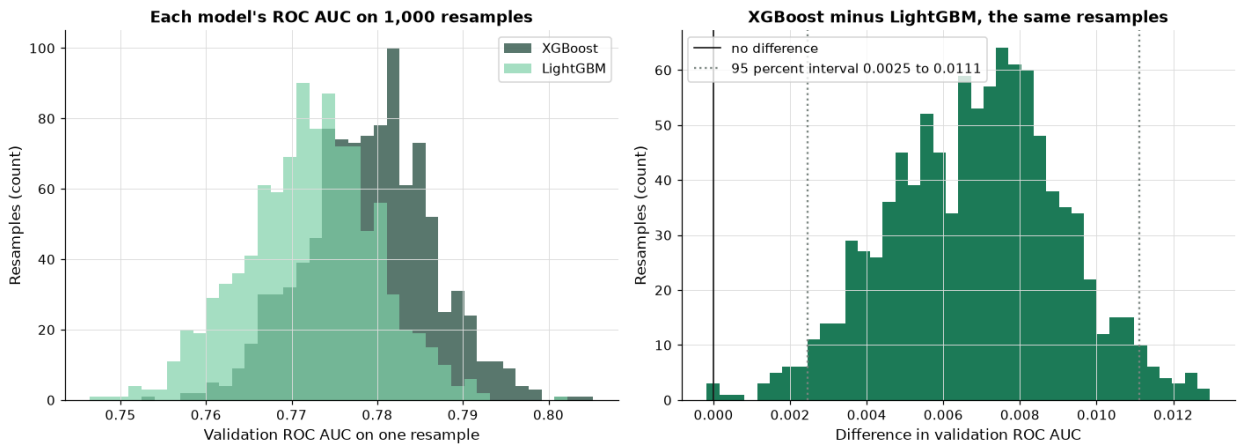
## 0.7650 to 0.7933

XGBoost, validation ROC AUC 0.7794

## 0.7572 to 0.7871

LightGBM, validation ROC AUC 0.7725

95 percent intervals from 1,000 resamples of the validation rows; nothing refitted




### The paired difference

XGBoost minus LightGBM: 0.0025 to 0.0111. XGBoost ranks higher on 99.9% of the resamples. Both models were scored on the same rows, so they move together.

# How out-of-fold thresholds work

### 1. Out-of-fold scores

each training row scored by a model that did not see it

### 2. Apply the rule

the highest threshold with recall at least 0.70

### 3. Report

compare_models on the validation rows

### 4. No choice there

the validation rows pick no threshold

### You control

The recall target and the folds.

### What comes out

One threshold per model and a table of validation results at those thresholds.

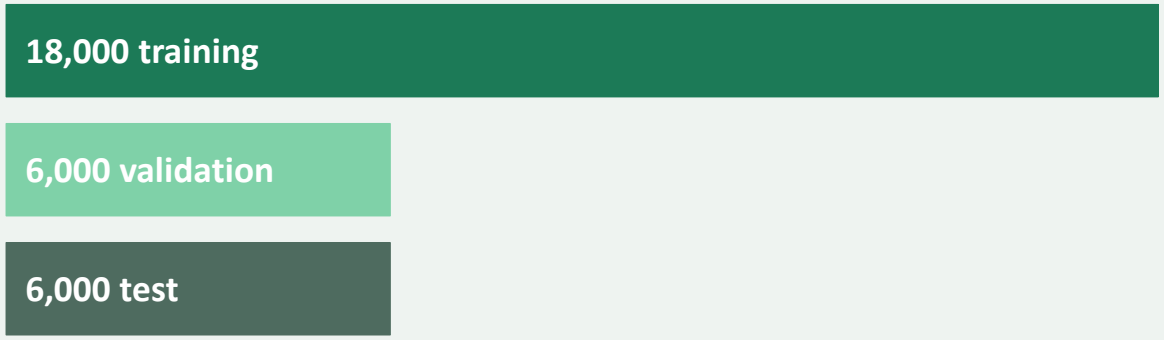
# Thresholds from the training rows

The highest threshold whose out-of-fold recall on the training rows is at least 0.70 (oof_threshold). The validation rows choose nothing.

| Validation rows | ROC AUC | Threshold | Recall | Precision |
|---|---|---|---|---|
| XGBoost | 0.7794 | 0.1812 | 0.6888 | 0.4115 |
| LightGBM | 0.7725 | 0.1778 | 0.6956 | 0.4043 |

**Recall lands near 0.70, not on it. Moving the threshold now would be choosing on the validation rows.**

# The case study, test rows closed




fit, tune, choose; 3,982 defaults, 22.12%

report only; 1,327 defaults

closed: never scored in Course 5

### Allowed names

model probability of default in this file

flagged at threshold t

the model's score (weighted or resampled)

### Never called

a credit decision

an approval or a decline

a risk grade or credit score

# Five models, fitted on the training rows

| Model | ROC AUC on its own training rows |
|---|---|
| Logistic regression (Course 4) | 0.7296 |
| Pruned tree (Course 4) | 0.7691 |
| Random forest, 200 trees | 0.9661 |
| XGBoost | 0.8496 |
| LightGBM | 0.9220 |

### Pruned tree

ccp_alpha 0.001, chosen by five folds of the training rows: 12 leaves.

**These numbers describe the fit. Nothing is chosen on them.**

# How a randomized search works

### 1. Write ranges

for the settings to search

### 2. Draw at random

one setting set per draw

### 3. Fit and score

five folds of the training rows per draw

### 4. Keep the best

its score is flattered by the choosing

### You control

The ranges, the number of draws, the scoring, and the seed.

### What comes out

A table of every draw, the best settings, and a best score that is not an honest estimate.

# A small search, and what best_score_ is not

| Rank | num_leaves | learning_rate | min_child_samples | fold mean | fold sd |
|---|---|---|---|---|---|
| 1 | 25 | 0.0153 | 333 | 0.7837 | 0.0018 |
| 2 | 18 | 0.0237 | 368 | 0.7837 | 0.0018 |
| 3 | 25 | 0.0139 | 150 | 0.7836 | 0.0022 |
| 4 | 7 | 0.0288 | 392 | 0.7829 | 0.0013 |

```
RandomizedSearchCV(..., n_iter=8, scoring="roc_auc", cv=mlkit.cv_folds(), random_state=SEED, n_jobs=1)
```

**best_score_ 0.7837: the best of 8 noisy fold means, on the folds that chose it. Not an honest score.**

# How the model is chosen

### 1. Out-of-fold scores

every training row scored once, unseen

### 2. Pool them

one ROC AUC over all the rows

### 3. Compare

the pooled score of each model

### 4. Carry one on

chosen before any validation row

### You control

The folds and the scoring.

### What comes out

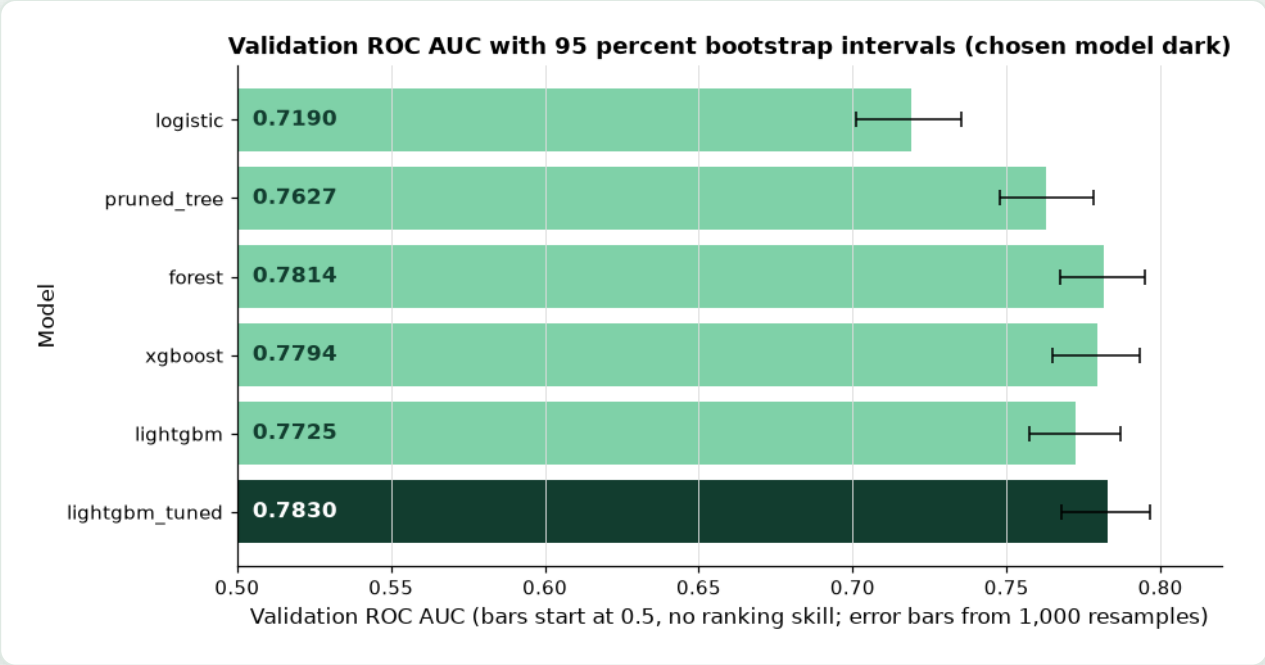
A table of thresholds and pooled scores, and the name of one model.

# The choice, inside the training rows

| Model | Threshold | Out-of-fold ROC AUC | Out-of-fold AP |
|---|---|---|---|
| Logistic regression (Course 4) | 0.2047 | 0.7273 | 0.5084 |
| Pruned tree (Course 4) | 0.4015 | 0.7603 | 0.5112 |
| Random forest, 200 trees | 0.1987 | 0.7754 | 0.5503 |
| XGBoost | 0.1812 | 0.7797 | 0.5542 |
| LightGBM | 0.1778 | 0.7743 | 0.5483 |
| LightGBM, tuned | 0.1890 | 0.7832 | 0.5488 |

**Chosen: LightGBM, tuned. Its settings were tuned on the same folds, so its score leans high.**

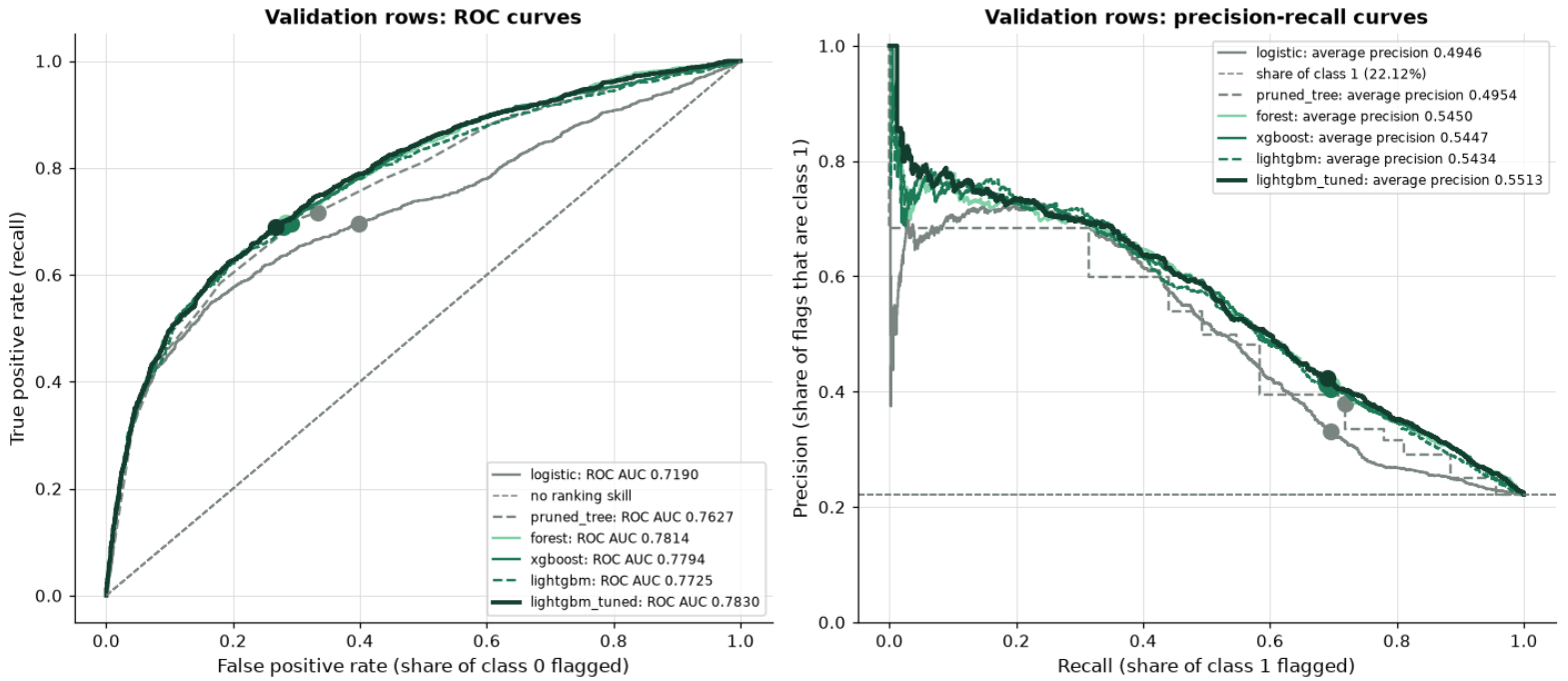
# The comparison on validation rows




| Validation | ROC AUC | Interval |
|---|---|---|
| logistic | 0.7190 | 0.7010 to 0.7352 |
| pruned_tree | 0.7627 | 0.7476 to 0.7784 |
| forest | 0.7814 | 0.7673 to 0.7948 |
| xgboost | 0.7794 | 0.7650 to 0.7933 |
| lightgbm | 0.7725 | 0.7572 to 0.7871 |
| lightgbm_tuned | 0.7830 | 0.7680 to 0.7968 |

**Accuracy sits beside the all-zero baseline (0.7788) and chooses nothing.**

# Six models, one chart each




**The four ensembles' curves lie almost on one line. Each dot is the model's out-of-fold threshold on rows that set nothing.**

# Paired, against the chosen model

Validation ROC AUC of lightgbm_tuned minus each model, on the same 1,000 resamples

| Model | Difference | Paired interval | Chosen ahead on | Reads as |
|---|---|---|---|---|
| Logistic regression (Course 4) | 0.0639 | 0.0509 to 0.0769 | 100% | beyond the noise |
| Pruned tree (Course 4) | 0.0203 | 0.0123 to 0.0279 | 100% | beyond the noise |
| Random forest, 200 trees | 0.0016 | -0.0040 to 0.0070 | 71% | within the noise |
| XGBoost | 0.0036 | -0.0006 to 0.0079 | 95% | within the noise |
| LightGBM | 0.0105 | 0.0053 to 0.0153 | 100% | beyond the noise |

**No interval here covers refitting with another seed.**

# How an importance table works

### 1. Shuffle a column

on rows the model was not fitted on

### 2. Record the drop

in ROC AUC: the permutation importance

### 3. Gain

the training loss each column's splits removed

### 4. Split count

how often each column is used

### You control

The rows, the score, and the number of shuffles.

### What comes out

A table of the measures for every column. It describes the model and says nothing about any person.

# What the chosen model leans on

| Column | Permutation | Split count | Split rank |
|---|---|---|---|
| PAY_0 | 0.0906 | 335 | 5 |
| LIMIT_BAL | 0.0239 | 660 | 1 |
| BILL_AMT1 | 0.0129 | 565 | 2 |

Permutation importance: the drop in validation ROC AUC when the column is shuffled. For LightGBM, feature_importances_ counts splits, not impurity.

### Allowed names

permutation importance; split count

the model leans on

association in this file

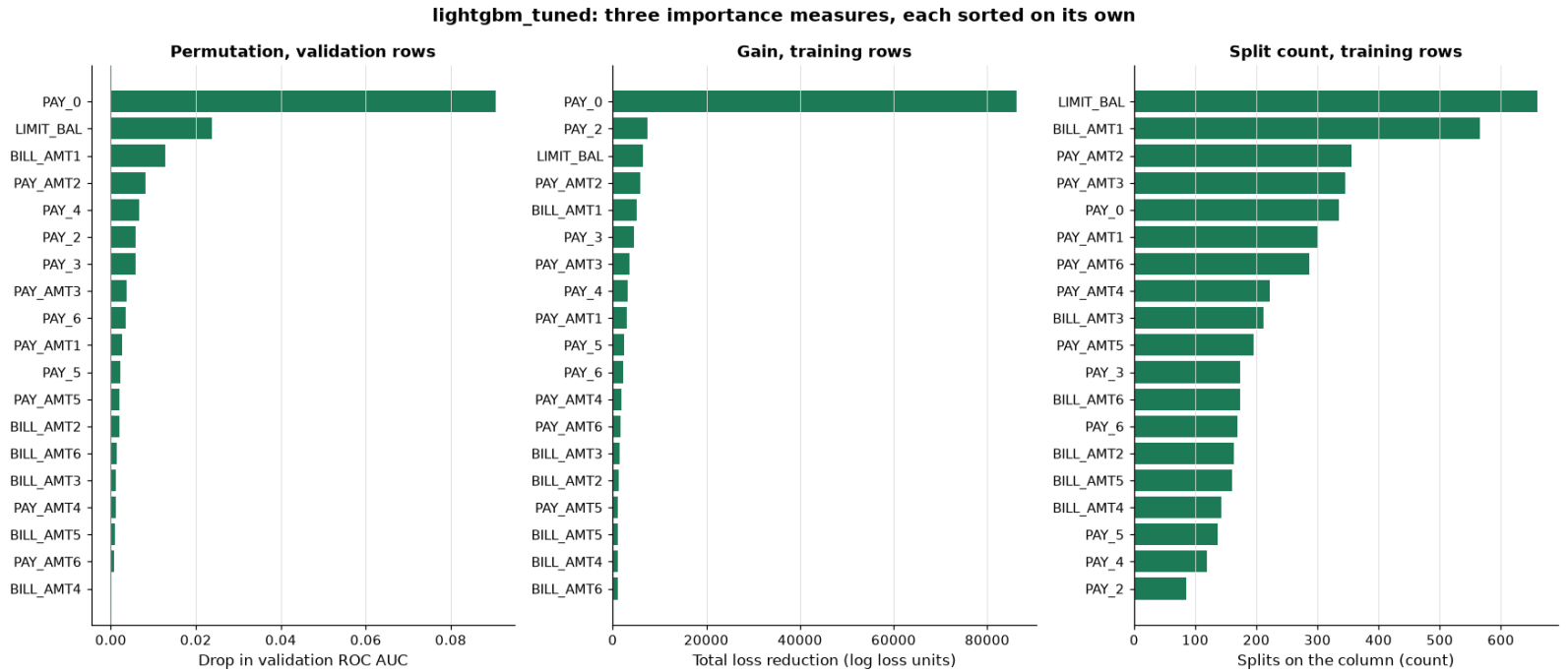
### Never called

driver, key driver

cause or determinant

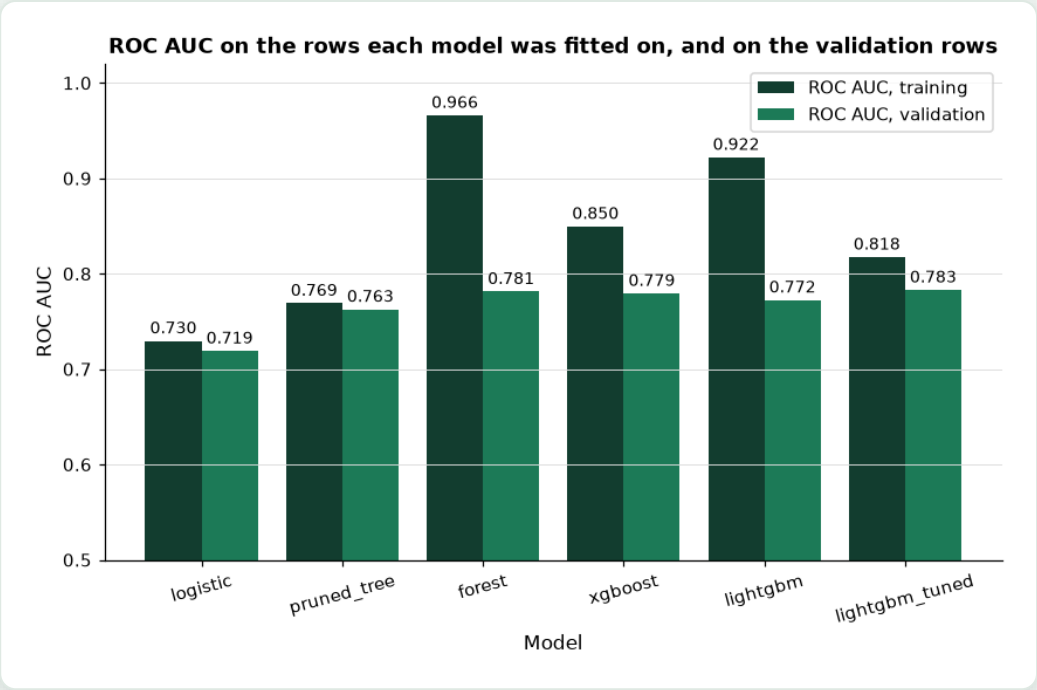
risk factor

# Three measures, three orders




**Permutation and gain put PAY_0 first, by far. By split count it ranks 5 of 19.**

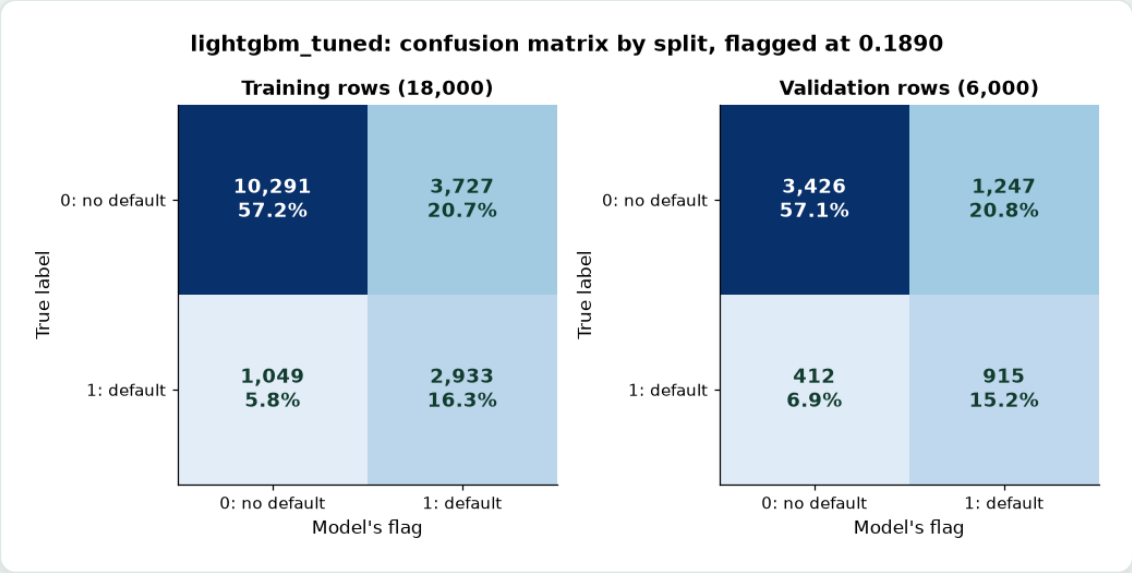
# Every model, both splits

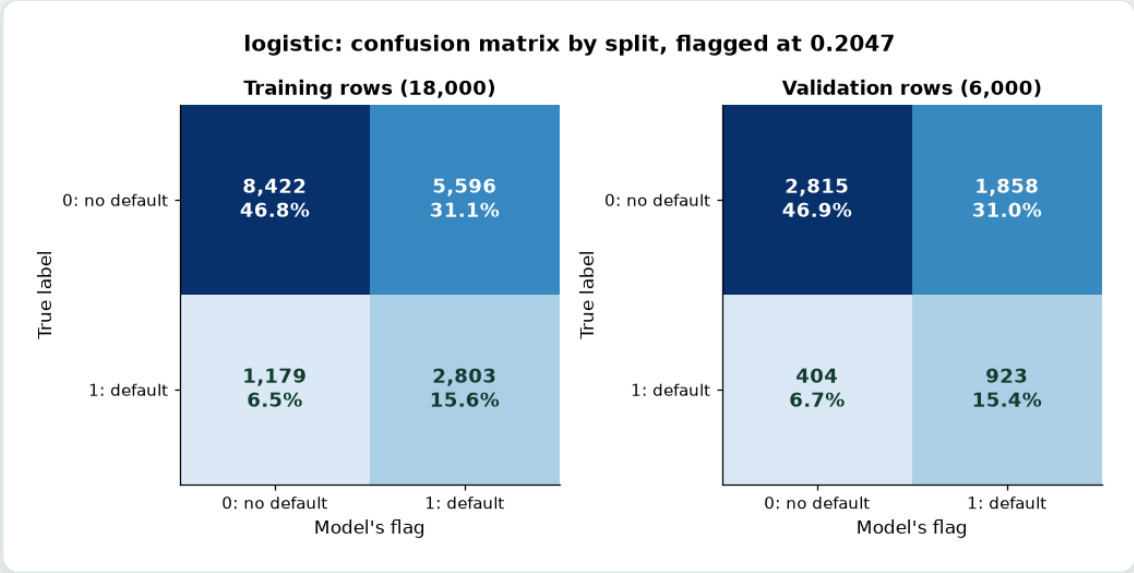



| Model | AUC train | AUC valid | Recall train | Recall valid |
|---|---|---|---|---|
| logistic | 0.7296 | 0.7190 | 0.7039 | 0.6956 |
| pruned_tree | 0.7691 | 0.7627 | 0.7328 | 0.7174 |
| forest | 0.9661 | 0.7814 | 0.9957 | 0.6971 |
| xgboost | 0.8496 | 0.7794 | 0.7870 | 0.6888 |
| lightgbm | 0.9220 | 0.7725 | 0.9214 | 0.6956 |
| lightgbm_tuned | 0.8175 | 0.7830 | 0.7366 | 0.6895 |

**The training columns describe the fit. The widest gap: forest.**

# The chosen model and the baseline, by split




**Training, then validation; each cell is a count and a share of that split's rows. True label down the side, the model's flag across.**

# What the model shows, and where it fails

**403 of 412**

missed defaults with a September status of 0 or below

**658 of 667**

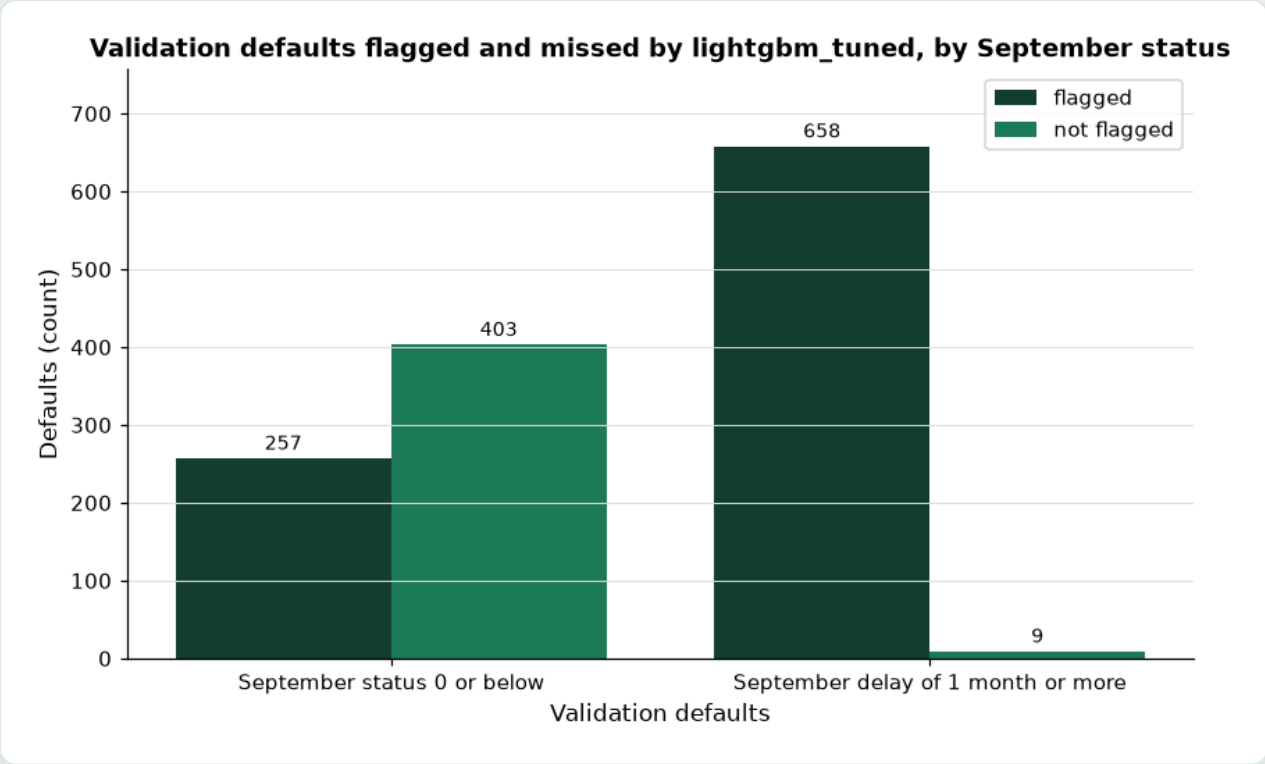
defaults with a recent delay, flagged

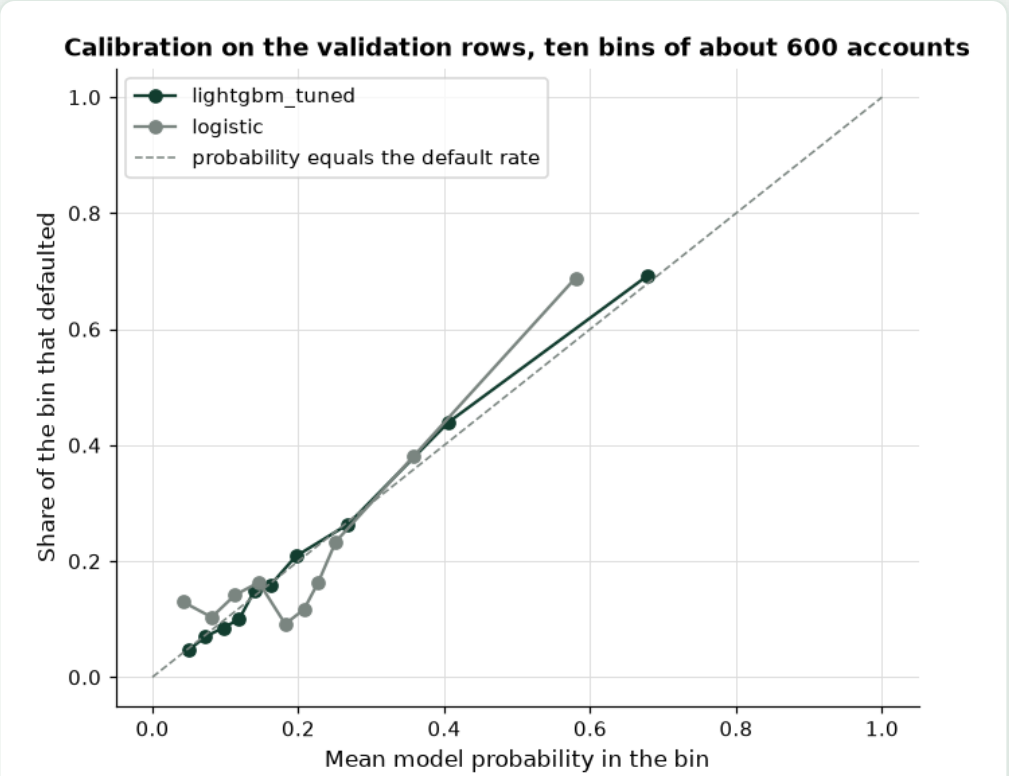
**257 of 660**

defaults without one, flagged

| | |
|---|---|
| **On rows** | 594 of 1,247 false flags were late in September and did not default. |
| **On the choice** | The paired intervals cannot separate it from Random forest, 200 trees or XGBoost. |
| **On the threshold** | Set for recall 0.70 on out-of-fold rows; validation is not a guarantee. |
| **On the data** | Taiwan, 2005; two codes undefined; how accounts were chosen is not stated. |

# Where the misses are, and how the probabilities read




**403 of the 412 missed defaults had no recorded September delay. The curve describes the probabilities; nothing corrects them today.**

# Excel side by side

### Excel side by side (tested in Excel)

| | |
|---|---|
| `=PERCENTILE.INC(A2:A1001,0.025)` | 0.7650: XGBoost's interval, lower end |
| `=PERCENTILE.INC(A2:A1001,0.975)` | 0.7933: upper end |
| `=PERCENTILE.INC(C2:C1001,0.025)` | 0.0025: the paired interval, lower end |
| `=PERCENTILE.INC(C2:C1001,0.975)` | 0.0111: upper end |
| `=COUNTIF(C2:C1001,">0")/COUNT(C2:C1001)` | 0.9990: share where XGBoost ranks higher |
| `=INDEX(A2:A7,MATCH(MAX(C2:C7),C2:C7,0))` | lightgbm_tuned: the model 10.7 chose |

**No forest, booster, search, or seeded bootstrap in Excel. It reads the CSV and the workbook (sheets comparison, intervals, thresholds); every number within 1e-12.**

# Save the day: closing week 2

### 1. Week 1

files cut back to week 1, committed, tag week1

### 2. Today

today's module and tests

### 3. Log

results.csv: six cv rows, six validation rows

### 4. Tag

git tag week2

```
git diff --stat week1 week2
3 files changed, 356 insertions(+)
```

**38 tests pass at the end of today. A tag works anywhere a commit does.**

# The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

## Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

## Code back

Written for the course in the style of an AI answer. Read it.

→

3

## Test first

Assert what the docstring promises. Run it: it fails.

→

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: a new train_test_split calls 6,000 rows fresh: 1,195 are closed test rows, and the ROC AUC (0.7903) looks better.**

*Code that runs is not the same as code that is right.*

# What you can do now

- Put a bootstrap interval on a validation score
- Compare two models by the paired difference
- Choose everything inside the training rows
- Say why best_score_ is not an honest score
- Run a case study with the test rows closed

**Week 3, Day 1** Randomized search and honest tuning